

Cheat sheet · Módulo 10: Deploy com Render

Resumo para consulta rápida. A explicação completa está nos slides e nos arquivos desta pasta.

O caminho de uma requisição

Nome (`api.meusite.com`) → DNS devolve o IP → conexão **HTTPS** (dados criptografados) → servidor → resposta.

Checklist antes do deploy

- ☐ Porta: `app.listen(process.env.PORT || 3000)`
- ☐ `"start": "node src/server.js"` no `package.json`
- ☐ Dependências de produção em `dependencies` (não em `devDependencies`)
- ☐ `.env` no `.gitignore` e `.env.example` no repositório
- ☐ Rota de saúde:

```
app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok" })
})
```

- ☐ Atlas: Network Access liberado para o Render (`0.0.0.0/0` , com senha forte)

Opção 1: render.yaml (Blueprint)

```
services:
  - type: web
    name: api-banco-digital
    runtime: node
    plan: free
    buildCommand: "npm install"
    startCommand: "npm start"
    autoDeploy: true
    envVars:
      - key: MONGO_URI
        sync: false # você digita o valor no painel
      - key: JWT_SECRET
        generateValue: true # o Render gera um valor aleatório
```

No painel: **New** → **Blueprint** → escolha o repositório.

Opção 2: configuração manual

New → **Web Service** → repositório → Build Command `npm install` → Start Command `npm start` → Environment: `MONGO_URI` , `JWT_SECRET` .

Depois de publicar

- Abra `https://sua-api.onrender.com/health` → `{"status":"ok"}` .
- Teste as rotas no Postman trocando `localhost:3000` pela URL pública.
- Algo errado? Aba **Logs** do serviço.
- Com `autoDeploy` , cada `git push` na `main` publica de novo.

Local x produção

	Local	Render
Porta	3000	<code>process.env.PORT</code>
Variáveis	arquivo <code>.env</code>	painel Environment
Sistema	Windows/Mac/Linux	Linux (maiúsculas importam!)
Endereço	<code>http://localhost</code>	<code>https://...onrender.com</code>

Plano free: cold start

Sem tráfego por cerca de 15 minutos, o serviço dorme. A primeira requisição acorda o servidor e pode levar perto de um minuto. O frontend precisa mostrar "carregando".

Frontend no navegador (CORS)

```
// npm install cors
import cors from "cors"
app.use(cors()) // permite que páginas de outros domínios chamem a API
```

Erros comuns

Sintoma	Causa
Deploy falha no build	pacote faltando em <code>dependencies</code>
Port scan timeout / sem resposta	porta fixa no <code>listen</code>
<code>MONGO_URI</code> undefined	variável não cadastrada ou com outro nome
Cannot find module './Config.js'	nome com maiúscula diferente (Linux diferencia)
Conexão com o banco expira	IP não liberado no Atlas

Segurança em produção

Segredos só no painel · `JWT_SECRET` longo e aleatório · nada de `--inspect` · respostas de erro sem detalhes internos.

LionsDev • Professor Nicolas Cardoso Motta
Cheat sheet • Módulo 10